```
function insert(node,
s)                                                  #1
  if s == “” then
                                                  #2
    node.keyNode ← true
  else
    (edge, commonPrefix, sSuffix, edgeSuffix) ←
matchEdge(node, s)   #4
    if edge == null then
         #5
      this.children[s[0]] ← new RTEdge(s, new
Node(true))
    elif edgeSuffix == “”
then                                              #6
      insert(edge.destination, sSuffix)
    else
      bridge ← new
Node(false)                                         #7
      this.children[s[0]] ← new RTEdge(commonPrefix,
bridge)         #8
      bridge.children[edgeSuffix[0]] ←
          new RTEdge(edgeSuffix,
edge.destination)                   #9
      insert(bridge,
sSuffix)                                          #10
```

#1 Method `insert` takes a trie node and the string key `s` to be inserted; it returns nothing but has side effects on the trie. Again, we assume that `node` is never `null`: this is a reasonable assumption if this method is implemented as a private method, internally called by the trie’s API `search` method.

#2 Check if the string searched is empty: if it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.